

IcomRigControl

Master Reference — CLAUDE.md Rules · wfview Feature Inventory ·
IC-7300 & IC-7300MK2 CI-V Protocol · C# Architecture

Jim (KE4CON) · .NET 10 · Avalonia 11 · macOS / Windows / Linux / Raspberry Pi

Contents

- Part 1 — CLAUDE.md Rules (copy into project root)
 - Part 2 — How to Create, Edit & Use CLAUDE.md
 - Part 3 — wfview Feature Inventory (complete)
 - Part 4 — IC-7300 & IC-7300MK2 CI-V Protocol Reference
 - 4A — Data Frame Format
 - 4B — Complete Command Table
 - 4C — Data Encoding Formats
 - Part 5 — C# CI-V Engine: Class Structure & Transceiver Model
 - Part 6 — Project Architecture & Build Phases
 - Appendix A — IC-7300 vs IC-7300MK2 Differences
 - Appendix B — wfview vs IcomRigControl Feature Comparison
-

How to use this document: Part 1 is a living spec — update it as the project evolves. Parts 3 and 4 are reference material to consult while building each feature. Part 5 is the day-one architecture. Parts stay self-contained so you can open to any section cold.

Part 1 — CLAUDE.md Rules

Copy the text below verbatim into a file named CLAUDE.md at the root of the IcomRigControl repository. Every Claude tool reads it at the start of every session.

```
# IcomRigControl – Project Rules (CLAUDE.md)

## Project Identity
Name: IcomRigControl
Author: Jim, KE4CON
Language: C# (.NET 10)
UI Framework: Avalonia 11 (cross-platform – macOS, Windows, Linux, Raspberry Pi)
Target Radios: Icom IC-7300 (address 94h) and IC-7300MK2 (address B6h)
Connection: USB serial via System.IO.Ports (115200 baud default); TCP planned for v2

## Architecture Layers (never mix concerns across layers)
Layer 1 – CivEngine: Raw CI-V framing, serial port I/O, BCD encode/decode. No UI, no radio model.
Layer 2 – RigModel: Transceiver class exposing clean C# properties and events. Consumes CivEngine only.
Layer 3 – Services: Logger, EMMCOM bridge, APRS beacon. Consume RigModel only.
Layer 4 – UI: Avalonia views and view-models. Consume Services and RigModel only.

## Coding Standards
- C# 12 features are fine (.NET 10 target)
- async/await everywhere I/O is involved – no blocking calls on the UI thread
- CancellationToken passed through all async paths
- Nullable reference types enabled
- Records for immutable data (CivFrame, MeterReading, ConditionsSnapshot)
- No magic numbers – all CI-V command bytes in CivCommands.cs
- All serial port access via ICivTransport interface to allow mocking in tests
- Unit tests required for: BCD encode/decode, frame builder, frame parser

## Feature Priorities (build in this order)
Phase 1: CI-V engine + serial + frequency read/set + mode read/set
Phase 2: Meter polling (S-meter, SWR, ALC, power, voltage, current)
Phase 3: Avalonia UI – main panel with frequency display, mode selector, meter gauges
Phase 4: Memory bulk editor (read all 99 channels, edit in DataGrid, write back)
Phase 5: Activity logger (CSV output, frequency/mode/meter timestamped)
Phase 6: EMMCOM dashboard integration
Phase 7: APRS beacon via CrossPlatformAPRS bridge
Phase 8: Spectrum scope capture and waterfall display
Phase 9: Remote/network mode (headless Pi server + TCP client)
Phase 10: Remote audio

## What NOT to do
- Do not implement features out of phase order without explicit instruction
- Do not add NuGet packages without listing them here first
- Do not put CI-V logic in ViewModels
- Do not put UI code in CivEngine or RigModel
- Do not use Thread.Sleep – use Task.Delay with CancellationToken
- Do not swallow exceptions silently

## Radio Addresses
IC-7300: 0x94 (controller default: 0xE0)
```

IC-7300MK2: 0xB6 (controller default: 0xE0)

Related Projects

- CrossPlatformAPRS (KE4CON/CrossPlatformAPRS): APRS beacon target for Phase 7
- EMMCOM Field Comms Server: dashboard integration target for Phase 6

Session Start Checklist

1. Read this file
2. Confirm which Phase is active
3. Check that the layer being touched matches the Phase
4. Do not refactor other layers unless the current Phase requires it

Part 2 — How to Create, Edit & Use CLAUDE.md

2.1 Creating the file

1. Open Terminal and cd into your IcomRigControl project folder.
2. Open in TextEdit (iCloud-safe): `open -e CLAUDE.md`
3. Paste Part 1 content and save with **■S**.
4. Commit: `git add CLAUDE.md && git commit -m "Add CLAUDE.md project rules"`

2.2 How each tool reads it

Claude Code (terminal): reads CLAUDE.md automatically every time you run `claude` in the project folder.

VS Code assistant: reads the file when you start a conversation in the project context.

This chat: paste the relevant sections when starting a planning session.

2.3 Editing the rules

Open with: `open -e CLAUDE.md` — then edit, save, and commit. Most common updates: mark a phase active, add a NuGet package entry, record a design decision.

2.4 When Claude drifts from the rules

Paste this into the assistant: "Please re-read CLAUDE.md and confirm which phase is active before proceeding."

Part 3 — wfview Feature Inventory

Source: wfview v2.01 (December 2024), GPL v3. Use this as a feature checklist — anything wfview does, IcomRigControl can replicate via the same published CI-V protocol. Reimplementing in C# from scratch means no license obligation.

3.1 Radio Connection

Feature	wfview Implementation	Phase
USB/Serial CI-V	System serial port, configurable baud up to 115200	Phase 1
Network (LAN/WiFi)	OEM Icom network protocol for radios with built-in LAN	Phase 9
Auto port detection	Scans available serial ports, selects first match	Phase 1
Baud rate selection	Dropdown 1200-115200	Phase 1
Radio model selection	Rig files define per-model capabilities	Phase 1 (enum)
Set radio clock on connect	Syncs PC time to radio on connection	Phase 1 (easy add)

3.2 Frequency & VFO Control

Feature	wfview Notes	Phase
Read/set VFO A frequency	CI-V 03h/05h, BCD encoded	Phase 1
Read/set VFO B frequency	CI-V 25h (unselected VFO)	Phase 1
VFO A/B swap	CI-V 07h B0h	Phase 1
VFO A=B copy	CI-V 07h A0h	Phase 1
Select VFO A or B	CI-V 07h 00h / 07h 01h	Phase 1
Tuning step set/read	CI-V 10h, 8 step sizes	Phase 1
Band stacking registers	CI-V 1Ah 01h, 3 registers per band	Phase 4
RIT frequency set/read	CI-V 21h 00h, +/- offset	Phase 3
RIT on/off	CI-V 21h 01h	Phase 3
Delta-TX on/off	CI-V 21h 02h	Phase 3
Split on/off	CI-V 0Fh	Phase 3
Transmit frequency read	CI-V 1Ch 03h	Phase 2
Band switcher UI	One-click band buttons	Phase 3
Frequency direct entry	Keyboard input box	Phase 3

3.3 Mode & Filter Control

Feature	wfview Notes	Phase
Read/set operating mode	LSB USB AM CW CW-R RTTY RTTY-R FM DV via CI-V 04h/06h	Phase 1
Data mode on/off	CI-V 1Ah 06h (USB-D, AM-D etc.)	Phase 1

Feature	wfview Notes	Phase
Filter selection FIL1/2/3	Included in mode command	Phase 1
IF filter width set/read	CI-V 1Ah 03h, per-mode widths	Phase 3
Passband tuning PBT1/PBT2	CI-V 14h 07h / 14h 08h	Phase 3
AGC speed set/read	CI-V 16h 12h (FAST/MID/SLOW)	Phase 3
IF filter shape SHARP/SOFT	CI-V 16h 56h	Phase 3
TX passband width	CI-V 1Ah 05h 00 14-17 (WIDE/MID/NAR)	Phase 3

3.4 Receive Features

Feature	wfview Notes	Phase
S-meter read	CI-V 15h 02h — 00 00=S0, 01 20=S9, 02 41=S9+60dB	Phase 2
Squelch status read	CI-V 15h 01h and 15h 05h	Phase 2
AF gain set/read	CI-V 14h 01h	Phase 3
RF gain set/read	CI-V 14h 02h	Phase 3
Squelch level set/read	CI-V 14h 03h	Phase 3
Preamplifier set/read	CI-V 16h 02h (OFF/P.AMP1/P.AMP2)	Phase 3
Attenuator set/read	CI-V 11h (0/20dB)	Phase 3
Noise reduction on/off + level	CI-V 16h 40h on/off; 14h 06h level	Phase 3
Noise blanker on/off + level	CI-V 16h 22h on/off; 14h 12h level	Phase 3
Auto notch on/off	CI-V 16h 41h	Phase 3
Manual notch on/off + position	CI-V 16h 48h / 14h 0Dh	Phase 3
APF on/off + peak	CI-V 16h 32h / 14h 05h	Phase 3
Twin Peak Filter on/off	CI-V 16h 4Fh	Phase 3
IP Plus on/off	CI-V 16h 65h	Phase 3
Lock on/off	CI-V 16h 50h	Phase 3

3.5 Transmit Features

Feature	wfview Notes	Phase
PTT on/off	CI-V 1Ch 00h	Phase 1
RF power set/read	CI-V 14h 0Ah	Phase 2
Mic gain set/read	CI-V 14h 0Bh	Phase 3
Speech compressor on/off + level	CI-V 16h 44h / 14h 0Eh	Phase 3
VOX on/off + sensitivity	CI-V 16h 46h / 14h 16h	Phase 3
TX inhibit on/off	CI-V 16h 66h	Phase 3
ALC meter read	CI-V 15h 13h	Phase 2
COMP meter read	CI-V 15h 14h	Phase 2
Po (power output) meter read	CI-V 15h 11h	Phase 2
SWR meter read	CI-V 15h 12h	Phase 2
Vd (supply voltage) read	CI-V 15h 15h	Phase 2
Id (current draw) read	CI-V 15h 16h	Phase 2
MOD input source select	CI-V 1Ah 05h 00 84-85h	Phase 3

3.6 Repeater & Tones

Feature	wfview Notes	Phase
Repeater tone on/off	CI-V 16h 42h	Phase 3
Tone squelch on/off	CI-V 16h 43h	Phase 3
CTCSS/TSQ L tone freq set/read	CI-V 1Bh 00h / 1Bh 01h	Phase 3
Repeater duplex/offset	CI-V 0Fh + 1Ah 05h 00 34h	Phase 3

3.7 Memory Management

Feature	wfview Notes	Phase
Switch to memory mode	CI-V 08h	Phase 4
Select memory channel 01-99	CI-V 08h 00h + channel number	Phase 4
Store VFO to memory	CI-V 09h	Phase 4
Copy memory to VFO	CI-V 0Ah	Phase 4
Clear memory channel	CI-V 0Bh	Phase 4
Read memory content	CI-V 1Ah 00h — freq, mode, name, tones, split	Phase 4
Write memory content	CI-V 1Ah 00h with all fields	Phase 4
Memory name (up to 16 chars)	Included in 1Ah 00h, ASCII encoded	Phase 4
Bulk memory editor	Table editor: read all, edit, write all	Phase 4
Programmed scan edges P1/P2	CI-V 08h 01 00 / 01 01	Phase 5

3.8 Scanning

Feature	wfview Notes	Phase
Programmed scan start/stop	CI-V 0Eh 01h / 00h	Phase 5
Memory scan	CI-V 0Eh 22h	Phase 5
Select memory scan	CI-V 0Eh 23h	Phase 5
Delta-F scan	CI-V 0Eh 03h	Phase 5
Tone scan	Automated CTCSS search	Phase 5

3.9 Spectrum Scope

Feature	wfview Notes	Phase
Scope on/off	CI-V 27h 10h	Phase 8
Waveform data output enable	CI-V 27h 11h — streams 475 data points per sweep	Phase 8
Scope mode (Center/Fixed/Scroll)	CI-V 27h 14h	Phase 8
Span setting (Center mode)	CI-V 27h 15h — 2.5kHz to 500kHz	Phase 8
Hold on/off	CI-V 27h 17h	Phase 8
Reference level	CI-V 27h 19h, +/-20dB in 0.5dB steps	Phase 8
Sweep speed	CI-V 27h 1Ah (FAST/MID/SLOW)	Phase 8
VBW (Video Bandwidth)	CI-V 27h 1Dh (NARROW/WIDE)	Phase 8
Scope during TX	CI-V 27h 1Bh	Phase 8
Overnight band logger	Custom — log waveform data to CSV	Phase 8

3.10 CW Features

Feature	wfview Notes	Phase
CW message send (30 chars)	CI-V 17h with ASCII codes, FFh=stop	Phase 3
Keying speed set/read	CI-V 14h 0Ch (6-48 WPM)	Phase 3
CW pitch set/read	CI-V 14h 09h (300-900 Hz)	Phase 3
Break-in on/off + type	CI-V 16h 47h (OFF/SEMI/FULL)	Phase 3
Keyer memory channels M1-M8	CI-V 1Ah 02h, up to 70 chars each	Phase 3
APF for CW	CI-V 16h 32h	Phase 3

3.11 Radio Sharing & Integration

Feature	wfview Implementation	IcomRigControl Equivalent
Hamlib rigctld server (port 4533)	Built-in rigctl emulation; N1MM+, fldigi, MacLoggerDX connect	Phase 9 — expose TCP API

Feature	wfview Implementation	IcomRigControl Equivalent
Virtual serial port	Win: loopback; Linux/Mac: pseudo-terminal	Phase 9 — virtual port bridge
Raw TCP server	Used by N1MM+ and RUMLogNG	Phase 9
Multiple program connection	Up to N programs via rigctld	Phase 9
Audio sharing (network radios)	OGG/ADPCM/LPCM streams	Phase 10
Stream Deck controller	USB controller integration	v2

3.12 System & Diagnostics

Feature	wfview Notes	Phase
Radio ID read	CI-V 19h	Phase 1
Power on/off	CI-V 18h 01h / 18h 00h	Phase 1
Set radio clock	CI-V 1Ah 05h 01 32-33h	Phase 1
Supply voltage read (Vd)	CI-V 15h 15h — 0-16V	Phase 2
Current draw read (Id)	CI-V 15h 16h — 0-25A	Phase 2
Backlight brightness set/read	CI-V 1Ah 05h 01 15h	Phase 3
TX timeout timer set	CI-V 1Ah 05h 00 32h	Phase 3
IP address read/set over CI-V	CI-V 1Ah 05h 01 01-06h (MK2 only)	Phase 9
NTP sync (MK2 only)	CI-V 1Ah 07h / 1Ah 08h	Phase 1

Part 4 — IC-7300 & IC-7300MK2 CI-V Protocol Reference

Source: IC-7300MK2 CI-V Reference Guide, Icom Inc. IC-7300 command set is identical except address 94h vs B6h and a small number of MK2 additions (see Appendix A).

4A — Data Frame Format

Byte	Value	Description
1	FEh	Preamble (fixed)
2	FEh	Preamble (fixed)
3	B6h / 94h	Radio address (MK2=B6h, IC-7300=94h)
4	E0h	Controller (PC) address (default E0h)
5	Cmd	Command number
6	Subcmd	Subcommand (if required by command)
7..n	Data	BCD or binary data bytes (if required)
n+1	FDh	End of message (fixed)

Radio to controller OK: FE FE E0 [radio addr] FBh FDh

Radio to controller NG: FE FE E0 [radio addr] FAh FDh

Frequency BCD: 5 bytes, reversed digit order. 14.074.000 Hz = 00 40 07 14 00 (each nibble = one decimal digit, least significant first).

4B — Complete Command Table

Cmd	Subcmd	Description	R/W
00h	—	Set and output current operating frequency (transceive mode)	W
01h	—	Set and output current operating mode (transceive mode)	W
02h	—	Read current upper and lower band edge frequencies	R
03h	—	Read current operating frequency	R
04h	—	Read current operating mode	R
05h	—	Set current operating frequency	W
06h	—	Set operating mode (with filter selection)	W
07h	00h	Select VFO A	W
07h	01h	Select VFO B	W
07h	A0h	Equalize VFO A and VFO B (A=B)	W
07h	B0h	Exchange VFO A with VFO B (swap)	W
08h	—	Switch to Memory mode	W
08h	00 01..99	Select memory channel 01-99	W
08h	01 00	Select scan edge P1	W
08h	01 01	Select scan edge P2	W

Cmd	Subcmd	Description	R/W
09h	—	Store VFO to selected memory channel	W
0Ah	—	Copy selected memory channel to VFO	W
0Bh	—	Clear selected memory channel	W
0Eh	00h	Stop current scan	W
0Eh	01h	Start programmed or memory scan	W
0Eh	03h	Start delta-F scan	W
0Eh	22h	Start memory scan	W
0Eh	23h	Start select memory scan	W
0Eh	B0h	Release select memory channel	W
0Eh	B1h	Set channel as select memory	W
0Eh	D0h	Turn scan resume OFF	W
0Fh	—	Read split on/off status	R
0Fh	00h	Turn split OFF	W
0Fh	01h	Turn split ON	W
10h	00-08h	Set/read tuning step (8 sizes)	R/W
11h	00h/20h	Set/read attenuator (00=OFF, 20=20dB)	R/W
13h	00h	Announce S-meter + frequency + mode (voice)	W
14h	01h	Set/read AF gain	R/W
14h	02h	Set/read RF gain	R/W
14h	03h	Set/read squelch level	R/W
14h	05h	Set/read APF peak frequency	R/W
14h	06h	Set/read noise reduction level	R/W
14h	07h	Set/read PBT inner (PBT1) shift	R/W
14h	08h	Set/read PBT outer (PBT2) shift	R/W
14h	09h	Set/read CW pitch (300-900 Hz)	R/W
14h	0Ah	Set/read RF power level	R/W
14h	0Bh	Set/read microphone gain	R/W
14h	0Ch	Set/read CW keying speed (6-48 WPM)	R/W
14h	0Dh	Set/read manual notch position	R/W
14h	0Eh	Set/read speech compressor level	R/W
14h	0Fh	Set/read CW break-in delay	R/W
14h	12h	Set/read noise blanker level	R/W
14h	15h	Set/read monitor (sidetone) level	R/W
14h	16h	Set/read VOX sensitivity	R/W

Cmd	Subcmd	Description	R/W
14h	17h	Set/read anti-VOX sensitivity	R/W
15h	01h	Read squelch open/closed status	R
15h	02h	Read S-meter level (00 00=S0, 01 20=S9, 02 41=S9+60dB)	R
15h	11h	Read Po (power output) meter	R
15h	12h	Read SWR meter	R
15h	13h	Read ALC meter	R
15h	14h	Read COMP meter	R
15h	15h	Read Vd (supply voltage) meter	R
15h	16h	Read Id (current draw) meter	R
16h	02h	Set/read preamp (OFF/P.AMP1/P.AMP2)	R/W
16h	12h	Set/read AGC time constant (FAST/MID/SLOW)	R/W
16h	22h	Set/read noise blanker on/off	R/W
16h	32h	Set/read APF mode (OFF/WIDE/MID/NAR)	R/W
16h	40h	Set/read noise reduction on/off	R/W
16h	41h	Set/read auto notch on/off	R/W
16h	42h	Set/read repeater tone on/off	R/W
16h	43h	Set/read tone squelch on/off	R/W
16h	44h	Set/read speech compressor on/off	R/W
16h	46h	Set/read VOX on/off	R/W
16h	47h	Set/read CW break-in (OFF/SEMI/FULL)	R/W
16h	48h	Set/read manual notch on/off	R/W
16h	4Fh	Set/read Twin Peak Filter on/off	R/W
16h	50h	Set/read lock on/off	R/W
16h	56h	Set/read IF filter shape (SHARP/SOFT)	R/W
16h	65h	Set/read IP Plus on/off	R/W
16h	66h	Set/read TX inhibit on/off	R/W
17h	data	Send CW message (up to 30 chars ASCII, FFh=stop)	W
18h	00h	Turn transceiver OFF	W
18h	01h	Turn transceiver ON (requires multi-FEh preamble)	W
19h	00h	Read transceiver ID	R
1Ah	00h	Set/read memory channel content (freq/mode/name/tones)	R/W
1Ah	01h	Set/read band stacking register content	R/W
1Ah	02h	Set/read keyer memory content (M1-M8, 70 chars each)	R/W
1Ah	03h	Set/read IF filter width (per mode)	R/W

Cmd	Subcmd	Description	R/W
1Ah	04h	Set/read AGC time constant (detailed, per mode)	R/W
1Ah	05h	Set/read all SET menu items (200+ items via sub-address)	R/W
1Ah	06h	Set/read DATA mode and filter width	R/W
1Ah	07h	Initiate NTP server access (MK2 only)	W
1Ah	08h	Read NTP server access result (MK2 only)	R
1Bh	00h	Set/read repeater tone (CTCSS) frequency	R/W
1Bh	01h	Set/read tone squelch (TSQL) frequency	R/W
1Ch	00h	Set/read PTT (RX=00h, TX=01h)	R/W
1Ch	01h	Set/read antenna tuner status (OFF/ON/Tune)	R/W
1Ch	02h	Set/read XFC monitor on/off	R/W
1Ch	03h	Read transmit frequency	R
21h	00h	Set/read RIT frequency (+/- offset)	R/W
21h	01h	Set/read RIT on/off	R/W
21h	02h	Set/read delta-TX on/off	R/W
25h	—	Set/read unselected VFO frequency	R/W
26h	—	Set/read unselected VFO mode/data/filter	R/W
27h	00h	Output scope waveform data (475 points)	R
27h	10h	Set/read spectrum scope on/off	R/W
27h	11h	Set/read waveform data output to PC on/off	R/W
27h	14h	Set/read scope mode (Center/Fixed/Scroll)	R/W
27h	15h	Set/read scope span (Center mode, 2.5kHz-500kHz)	R/W
27h	17h	Set/read scope hold on/off	R/W
27h	19h	Set/read scope reference level (+/-20dB in 0.5dB)	R/W
27h	1Ah	Set/read scope sweep speed (FAST/MID/SLOW)	R/W
27h	1Bh	Set/read scope-during-TX on/off	R/W
27h	1Dh	Set/read VBW (Narrow/Wide)	R/W
27h	1Eh	Set/read scope fixed edge frequencies	R/W
28h	01-08h	Start Voice TX memory 1-8	W

4C — Key Data Encoding Formats

Frequency (5 bytes, BCD reversed):

Each byte holds two BCD digits (one per nibble), least significant first.

14,074,000 Hz: digits 0,0,0,0,4,7,0,4,1 LSB-first pairs per byte:

Byte 0: 0+0=0x00 Byte 1: 0+4=0x40 Byte 2: 7+0=0x07 Byte 3: 4+1=0x14 Byte 4: 0+0=0x00

Operating Mode codes (command 04h/06h):

Code	Mode	Code	Mode
00h	LSB	05h	FM
01h	USB	07h	CW-R
02h	AM	08h	RTTY-R
03h	CW	17h	DV (D-STAR)
04h	RTTY	—	—

Filter byte (follows mode byte in 06h command): 01h=FIL1, 02h=FIL2, 03h=FIL3.

S-Meter scale (read via 15h 02h):

Value	Level	Value	Level
00 00h	S0	01 20h	S9
00 20h	S1	01 40h	S9+10dB
00 40h	S2	01 60h	S9+20dB
00 60h	S3	01 80h	S9+30dB
00 80h	S4	01 A0h	S9+40dB
00 A0h	S5	01 C0h	S9+50dB
00 C0h	S6	02 41h	S9+60dB
00 E0h	S7	—	—
01 00h	S8	—	—

Continuous values (14h level commands): 00 00h=minimum, 02 55h=maximum.

Part 5 — C# CI-V Engine: Class Structure & Transceiver Model

5.1 Solution Layout

```
IcomRigControl.sln
IcomRigControl.CivEngine/
  CivCommands.cs      - All command byte constants
  CivFrame.cs         - Immutable record: parsed CI-V frame
  CivFrameBuilder.cs  - Build byte arrays for each command
  CivFrameParser.cs   - Parse incoming byte stream to CivFrame
  BcdCodec.cs         - Frequency BCD encode/decode
  ICivTransport.cs    - Interface: Open/Close/Write/DataReceived
  SerialCivTransport.cs - System.IO.Ports implementation
IcomRigControl.RigModel/
  Transceiver.cs      - Main class, consumes CivEngine
  TransceiverConfig.cs - Address, baud rate, model enum
  MeterSnapshot.cs    - Record: all meter values at one moment
  RadioModel.cs       - Enum: IC7300, IC7300MK2
IcomRigControl.Services/
  ActivityLogger.cs   - CSV frequency/mode/meter log
  MemoryManager.cs   - Bulk read/write of 99 channels
  EmmcomBridge.cs     - Push status to Field Comms Server
  AprsBridge.cs       - Beacon freq to CrossPlatformAPRS
  PresetManager.cs    - Stored rig-state profiles
IcomRigControl.UI/
  Avalonia desktop app - ViewModels + Views
IcomRigControl.Tests/
  BcdCodecTests.cs    - 14 tests PASSING
  FrameBuilderTests.cs - next milestone
  FrameParserTests.cs
CLAUDE.md
```

5.2 Transceiver.cs — The RigModel Core

```
public class Transceiver : INotifyPropertyChanged, IAsyncDisposable
{
    // Observable properties (all fire PropertyChanged)
    public long FrequencyHz { get; private set; } // VFO A
    public long FrequencyBHz { get; private set; } // VFO B
    public string Mode { get; private set; } = ;
    public int FilterNumber { get; private set; } // 1/2/3
    public bool DataModeActive { get; private set; }
    public bool PttActive { get; private set; }
    public bool SplitActive { get; private set; }
    public bool IsConnected { get; private set; }

    // Meter readings (updated by polling loop)
    public double SMeterDbm { get; private set; }
    public int SMeterS { get; private set; }
    public double RfPowerPercent { get; private set; }
    public double SwrRatio { get; private set; }
    public double AlcLevel { get; private set; }
    public double SupplyVoltage { get; private set; }
    public double CurrentDraw { get; private set; }

    // Events
```



```

public event EventHandler<MeterSnapshot>? MeterUpdated;
public event EventHandler<long>? FrequencyChanged;
public event EventHandler<string>? ModeChanged;
public event EventHandler<bool>? PttChanged;
public event EventHandler<byte[]>? ScopeDataReceived;

// Public async command methods
public Task ConnectAsync(CancellationTokens ct = default);
public Task DisconnectAsync();
public Task SetFrequencyAsync(long hz, CancellationTokens ct = default);
public Task SetModeAsync(string mode, int filter = 1, bool dataMode = false,
CancellationTokens ct = default);
public Task SetPttAsync(bool transmit, CancellationTokens ct = default);
public Task SetRfPowerAsync(double percent, CancellationTokens ct = default);
public Task<List<MemoryChannel>> ReadAllMemoriesAsync(CancellationTokens ct = default);
public Task WriteMemoryAsync(MemoryChannel ch, CancellationTokens ct = default);
}

```

5.3 MeterSnapshot Record

```

public record MeterSnapshot(
    DateTimeOffset Timestamp,
    long FrequencyHz,
    string Mode,
    double SMeterDbm,
    int SMeterS,
    double RfPowerPercent,
    double SwrRatio,
    double AlcLevel,
    double CompLevel,
    double SupplyVoltage,
    double CurrentDraw
);

```

This is what ActivityLogger writes to CSV, what the EMMCOM bridge pushes to the Field Comms Server, and what the APRS beacon reads frequency from. One clean record type flowing through all services.

Part 6 — Project Architecture & Build Phases

Phase	Goal	Milestone / Deliverable
1	CI-V Engine + Connection	CLI: type freq 14074000 -> radio QSYs. BCD tests pass. (IN PROGRESS)
2	Meter Polling	Console prints live S-meter, SWR, power, Vd, Id at 500ms. MeterSnapshot flowing.
3	Avalonia UI Panel	Desktop window: frequency display, mode buttons, meter gauges, PTT button.
4	Memory Bulk Editor	DataGrid shows all 99 channels. Edit in-place. Write all back with one button.
5	Activity Logger	CSV auto-created on connect. Every poll appends timestamp, freq, mode, all meters.
6	EMMCOM Bridge	Push MeterSnapshot JSON to Field Comms Server. Dashboard shows live rig status.
7	APRS Beacon	Beacon current frequency as APRS object to CrossPlatformAPRS via UDP.
8	Spectrum Scope	Waterfall from CI-V 27h 00h stream. Band activity recorder saves overnight CSV.
9	Remote / Network	Headless Pi mode: TCP server. Mac client connects over 44Net. rigctl port 4533.
10	Remote Audio	RX audio: NAudio (Windows) or AVFoundation (macOS). TX audio path.

Appendix A — IC-7300 vs IC-7300MK2 Differences

Item	IC-7300 (original)	IC-7300MK2
CI-V address	0x94 (94h)	0xB6 (B6h)
USB connector	USB Type-B	USB Type-C
Command reference	Chapter 19 of IC-7300 Full Manual	Dedicated CI-V Reference Guide (Oct 2025)
Network control	Not built-in (USB only)	LAN/network control built-in; CI-V over LAN supported
Network CI-V commands	N/A	1Ah 05h 01 00-13h (IP address, ports, DHCP, network name)
NTP sync	Not available	1Ah 07h / 1Ah 08h (initiate / read result)
External display	Not available	1Ah 05h 01 24-26h (enable, resolution, audio)
Waterfall via CI-V	Same 27h command set	Same 27h command set
Software handling	RadioModel.IC7300	RadioModel.IC7300MK2

Design rule: CivFrameBuilder checks TransceiverConfig.Model before issuing network-only commands. All other commands use identical builder methods.

Appendix B — wfview vs IcomRigControl Feature Comparison

Feature Area	wfview	IcomRigControl
Language / framework	C++ / Qt	C# / .NET 10 / Avalonia 11
License	GPLv3 (must open-source derivatives)	Your choice (MIT recommended)
Platforms	macOS, Windows, Linux	macOS, Windows, Linux, Raspberry Pi
Icom USB serial	Yes	Yes (Phase 1)
Icom LAN (OEM network)	Yes (IC-9700, IC-7610, IC-705)	Yes for MK2 (Phase 9)
Spectrum scope / waterfall	Yes, real-time	Yes (Phase 8)
Memory bulk editor	Yes, graphical	Yes, DataGrid (Phase 4)
rigctld server (Hamlib)	Yes, built-in port 4533	Yes (Phase 9)
Remote audio	Yes (network radios only)	Phase 10
Activity / band logger	No	Yes — CSV timestamped (Phase 5)
EMMCOM dashboard bridge	No	Yes (Phase 6)
APRS beacon	No	Yes via CrossPlatformAPRS (Phase 7)
Supply voltage / current log	Reads meters, no persistent log	Logged in MeterSnapshot CSV (Phase 5)
Field ops dashboard	No	Phase 6 (Vd, Id, ALC, power, SWR)
Band activity overnight logger	No	Phase 8 (scope waveform CSV)
Remote head over 44Net	Yes (wfview server on Pi)	Yes (Phase 9 headless mode on Pi)
CW sender UI	Yes	Phase 3
Multiple program sharing	Yes (rigctld)	Phase 9